Angriffe auf RSA

Grundlagenpraktikum zur IT-Sicherheit (WiSe 2023/2024)

Lehrstuhl für Netz- und Datensicherheit
Ruhr-Universität Bochum

Nico Steppeler
Matr.-Nr:1080 2221 3001
Nico.Steppeler@ruhr-uni-bochum.de

Sabri Öztürk
Matr.-Nr:1080 2122 7680
Sabri.Oeztuerk@ruhr-uni-bochum.de

27. April 2026

Zusammenfassung

In diesem Versuch wird ein Einblick in die Verschlüsselung und Signierung von Daten mit RSA erhalten. Zudem werden mehrere Angriffe auf verschiedene Weisen durchgeführt. So soll gezeigt werden, dass Angriffe sowohl mit physischem Zugriff, als auch rein digital geschehen können.

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Glossar 1

2 Aufbau **2**

3 Durchführung **3**

3.1 RSA-Verschlüsselung mit kleinem öffentlichen Exponenten 3

3.2 Fault-Angriff gegen RSA-CRT 5

3.3 Bug-Attack 6

4 Responsible disclosure **8**

5 Zusammenfassung und Fazit **9**

6 LLM **10**

1 Einleitung

In der heutigen Zeit werden mehr und mehr Daten digital ausgetauscht. Dies bietet viele Vor- aber auch Nachteile. Digitale Austausch ist schneller als analoge Austausch, jedoch werden die digitalen Dateien und Nachrichten sehr oft über öffentlichen Kanäle ausgetauscht, was die Sicherheit der digitalen Dateien und Nachrichten gefährdet.

Eine Unterschrift auf beispielsweise einer PDF Datei, kann einfach kopiert und von jemand anderem genutzt werden. Auch könnte der Inhalt verändert werden, die Unterschrift bleibt jedoch bestehen. Um auch bei digitalen Daten die Vertraulichkeit, Authentizität und Integrität zu gewährleisten gibt es Verschlüsselungen und digitale Signaturen.

Ein Beispiel, was gleich als Verschlüsselungsalgorithmus und Signaturalgorithmus eingesetzt werden kann ist RSA. Dieser Algorithmus wurde von Ronald L. Rivest, Adi Shamir und Leonard Adleman entwickelt und patentiert [3]. Es ist ein asymetrischer Algorithmus. Das bedeutet, jeder der den sogenannten *public key* (pk) hat, kann Daten verschlüsseln, jedoch nur der Empfänger mit dem *private key* diese entschlüsseln. Gleichermaßen kann nur der besitzter des *private key* Daten signieren, jedoch jeder andere diese signatur mit dem *public key* überprüfen.

Ein wichtiger Faktor bei dieser Verschlüsselung und Signierung ist, den *private key* geheim zu halten. Doch durch verschiedene Angriffe kann es Angreifern möglich werden, den Schlüssel zu berechnen oder wenigstens die möglichen Schlüssel eingrenzen, sodass ein Brute-Force-Angriff schneller funktionieren würde. Wie einige dieser Angriffe funktionieren, wird im Laufe diese Versuch erläutert.

1.1 Glossar

RSA asymmetrische Verschlüsselung entwickelt von Rivest, Shamir und Adleman

Signatur eine digitale Unterschrift für Dateien und Nachrichten

2 Aufbau

Die folgenden Versuche wurden im Netzlabor ID 2/168 unternommen.
Wir führen den Versuch zu zweit als Gruppe 43 am PC mit der IP-Adresse 192.168.1.42 durch.
Das Betriebsystem ist ein Ubuntu 22.04 LTS.

Es wurden verschiedene Programme und Befehle genutz. Diese sind:

Software	**Version**
Python	3.10.4

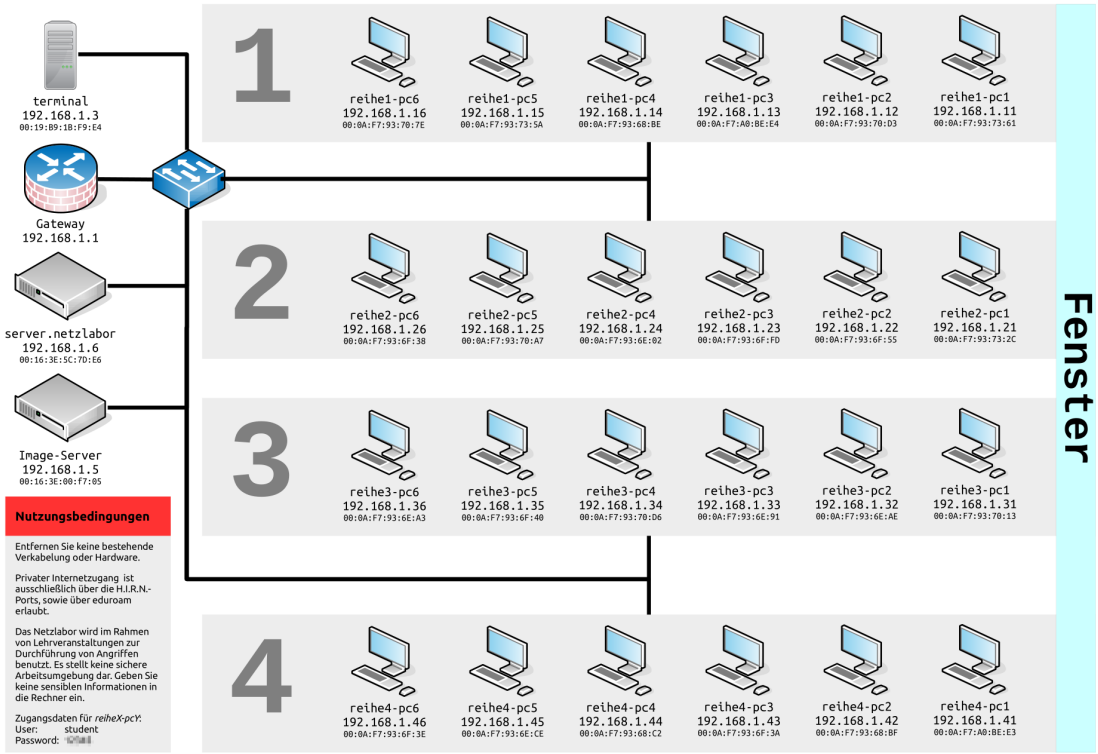

Abbildung 1: Netzplan Raum ID 2/168, Stand 01.05.2017

Die Aufgabenstellung [2] wurde bereutgestellt übe die Veranstaltungsseite [1]

3 Durchführung

Im Folgenden werden die einzelnen Aufgaben durchgeführt, erläutert und das Ergebnis präsentiert.

3.1 RSA-Verschlüsselung mit kleinem öffentlichen Exponenten

Im Verzeichnis '/home/student/Praktikum/rsa' befinden sich drei Dateien mit den base64-kodierten öffentlichen Schlüsseln 'costa.pk', 'gambino.pk' und 'zarella.pk'. Nun möchten wir die öffentlichen RSA-Parameter ausgeben lassen. Hierfür verwenden wir den OpenSSL-Befehl: *openssl rsa -inform PEM -pubin -in Name der Datei -text -noout*. Dabei wird OpenSSL für die Verarbeitung verwendet, 'rsa' ermöglicht den Zugriff auf den RSA-Algorithmus. Mit '-inform PEM' wird die Ausgabe im PEM-Format festgelegt, um kryptografische Daten anzuzeigen. 'pubin' gibt an, dass es sich um einen öffentlichen Schlüssel handelt. Mit '-in Name der Datei' wird explizit ausgewählt, von welcher Datei der Schlüssel ausgegeben werden soll. '-text' gibt an, dass die Ausgabe im Textformat erfolgen soll. 'noout' zeigt an, dass nur die Informationen zum Schlüssel angefordert werden, nicht jedoch der Schlüssel selbst. Alle Dateien verwenden denselben öffentlichen Schlüssel 'e', welcher den Wert 3 hat. Die Datei 'costa.pk' verwendet den Modulus 3748150753, 'gambino.pk' den Modulus 3012222769 und 'zarella.pk' den Modulus 3715158301.

```
student@Reihe4-PC2:~/Praktikum/rsa$ openssl rsa -inform PEM -pubin -in costa.pk -text -
    noout
Public-Key: (32 bit)
Modulus: 3748150753 (0xdf683de1)
Exponent: 3 (0x3)
student@Reihe4-PC2:~/Praktikum/rsa$ openssl rsa -inform PEM -pubin -in gambino.pk -text
    -noout
Public-Key: (32 bit)
Modulus: 3012222769 (0xb38adf31)
Exponent: 3 (0x3)
student@Reihe4-PC2:~/Praktikum/rsa$ openssl rsa -inform PEM -pubin -in zarella.pk -text
    -noout
Public-Key: (32 bit)
Modulus: 3715158301 (0xdd70d11d)
Exponent: 3 (0x3)
student@Reihe4-PC2:~/Praktikum/rsa$
```

Wir verfügen über die Chiffrate der Nachrichten, die an die drei Mafiosi verschlüsselt wurden. Um den zugehörigen Klartext m zu ermitteln, nutzen wir den Befehl *python3*, um Python zu starten und die Berechnungen durchzuführen. Als zusätzliches Hilfsmittel greifen wir auf das Tool `wolframalpha` zurück, um die Inversen ohne Reduzierung berechnen zu können. Zunächst geben wir die verfügbaren Informationen in die Variablen ein. Anschließend lassen wir den Modulus N berechnen, indem wir alle vorgegebenen Moduli multiplizieren. Die Berechnung der Teile der Moduli für den Chinesischen Restsatz N1, N2, N3 erfolgt, indem jeweils einer der Moduli ausgelassen wird. Anschließend haben wir die Inversen der Modulteile mithilfe von `wolframalpha` berechnet. Dann haben wir die vorgegebenen Chiffretexte in Variablen gespeichert. Die Nachricht mit dem Exponenten 3 haben wir gefunden, indem wir den Chinesischen Restsatz angewendet haben. Aus dem resultierenden Ergebnis haben wir die Kubikwurzel gezogen. Abschließend haben wir das Ergebnis mithilfe einer ASCII-Tabelle übersetzt. Das Ergebnis lautet 'Tre'.

```
student@Reihe4-PC2:~/Praktikum/rsa$ python3
Python 3.10.4 (main, Jun 29 2022, 12:14:53) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> e=3
>>> n1=3748150753
>>> n2=3012222769
>>> n3=3715158301
>>> N=n1*n2*n3
>>> N1=n2*n3
>>> N2=n1*n3
>>> N3=n1*n2
```

```
>>> print("inverse per wolframalpha")
inverse per wolframalpha
>>> print(N1)
11190884424711555469
>>> print(N2)
13924973383407350653
>>> print(N3)
11290265039831095057
>>> N1_inv=1449895280
>>> N2_inv=1256330352
>>> N3_inv=728516526
>>> c1=1786058302
>>> c2=521332323
>>> c3=2806221682
>>> m3=(c1*(N1_inv%n1)*N1+c2*(N2_inv%n2)*N2+c3*(N3_inv%n3)*N3)%N
>>> m3
5951225722511119297723 01
>>> m=pow(m3, 1/float(3))
>>> m
84114101
```

3.2 Fault-Angriff gegen RSA-CRT

Nun wollen wir Fault-Angriff gegen RSA-CRT durchführen. Dazu ist eine Webapplikation 'smartcard' vorgegeben, auf die simulierte Smartcard Nachrichten signalisiert werden kann. Modulus n ist als '288791' vorgegeben.
Als erstes erledigen wir das mathematische Vorarbeit. Dafür versuchen wir ein Rechenvorschrift vorzubereiten, das aus den CRT-Parameter mp, dp, mq und dq und sowie aus den Faktoren des RSA-Moduls n, p und q, die RSA-Signatur berechnet. Die Formel für die Signaturberechnung in dem Fall ist:

$$s = (q^{-1} mod p) * m_p^{d_p} * q + (p^{-1} mod q) * m_q^{d_q} * p$$

Teile der Formel können durch eine unbekannte X ersetzt werden. Wenn der Teil $(p^{-1} \text{mod } q) * m_q^{d_p}$ durch X ersetzt wird, kommt die Formel

$$s' = (q^{-1} mod p) * m_p^{d_p} * q + X * p$$

heraus. Mit $ggt(s - s', n)$ kommt die Primzahl p heraus, da die Formel entsprechend umgestellt werden kann. Aus dem Umstellen ergibt sich

$$\begin{aligned} s - s' &= ((q^{-1} mod p) * m_p^{d_p} * q + (p^{-1} mod q) * m_q^{d_q} * p) - ((q^{-1} mod p) * m_p^{d_p} * q + X * p) \\ &= (p^{-1} mod q) * m_q^{d_q} * p - X * p \\ &= ((p^{-1} mod q) * m_q^{d_q} - X) * p \end{aligned}$$

Da nun $s - s'$ ein vielfaches von p ist und $n = p * q$ gilt, ist $ggT(s - s', n) = ggT(xp, qp) = p$ mit $x = ((p^{-1} mod q) * m_q^{d_q} - X)$, da nur p und q Teiler von n sind. Durch n/p wird auch die Primzahl q gefunden. Wenn wir also für eine Nachricht verschiedene Signaturen erzeugen können, können wir daraus p und q ableiten. Um eine zweite Signatur erzeugen zu können, können unterschiedliche Störungen wie ein elektromagnetisches Feld, Licht- oder Hitzeeinwirkung oder Spannungsschwankungen eingesetzt werden, was mit einer andere Exponent als vorgesehene gerechnet wird. Die Sicherheit der RSA-Signatur hängt von der Geheimhaltung der Primzahlen p und q ab. Durch das zuvor erwähnte Rechenvorschrift kann RSA-Signatur gebrochen werden.
In nächsten Schritt wollen wir zuvor erwähnten Angriff durchführen. Wir haben uns die Zahl '3' vorgenommen die Signatur einmal zu Ende signieren lassen und einmal früher abgebrochen und somit haben wir zwei verschiedene Signaturwerte erhalten. Die Ergebnisse haben wir gespeichert.

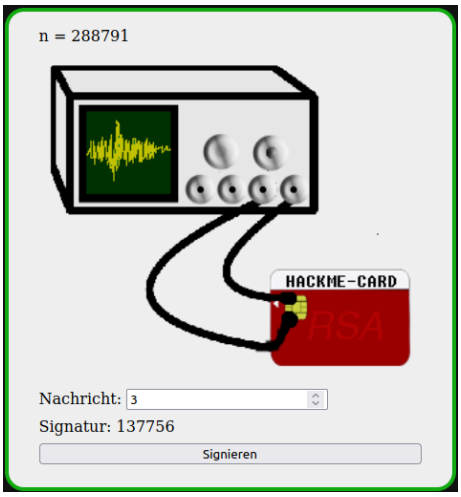

Abbildung 2: Erzeugte Signatur

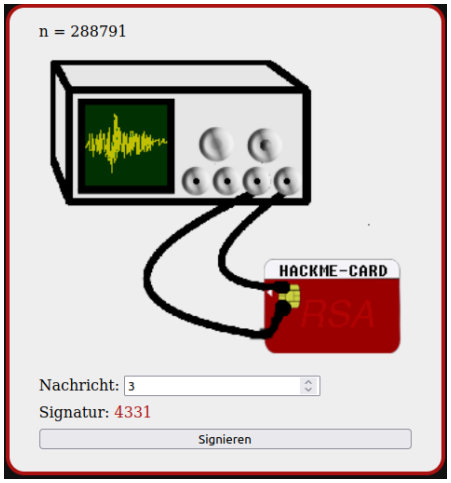

Abbildung 3: Durch Faultangriff erzeugte Signatur

Dann haben wir die Werte, die wir haben, in Variablen gespeichert. Daraufhin haben wir die oben beschriebene Angriffsvorbereitung in Python implementiert. Dadurch könnten wir die Primzahlen p und q herausfinden. Als Test haben wir p und q multipliziert und festgestellt, dass wirklich n herauskommt und somit p und q korrekt berechnet wurden.

```
student@Reihe4-PC2:~/Praktikum/rsa$ python3
Python 3.10.4 (main, Jun 29 2022, 12:14:53) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> n=288791
>>> m=3
>>> s=137756
>>> s_=4331
>>> def ggt(a, b):
...     while a % b != 0:
...             a, b = b, a%b
...     return b
...
>>> p=ggt(s-s_, n)
>>> q=n//p
>>> p
593
>>> q
487
>>> p*q
288791
```

3.3 Bug-Attack

In dieser Teil des Versuchs beschäftigen wir uns mit einer weiteren Angriff auf RSA-Signatur, die darauf abzielt, eine Nachricht, die verschiedene Signaturen erzeugt, zu finden, um daraus p und q abzuleiten. Vorgegeben ist eine Webanwendung, die zur festgelegten Parametern Modulus n ’207703’ und Exponent e ’17’ Signatur der Nachricht bildet. Dann konstruieren wir eine Nachricht m, in dem wir die Wurzel von n ziehen, was gerundet ’455’ ergibt, und dann aus den Speicherwörtern eine nachliegenste Zahl finden, die beim Signieren, verschiedene Ergebnisse liefert. Also die Binärzahlen eines Speicherwortes sollen als die ersten Ziffern der Nachricht m bleiben, und am Ende dieser Binärzahl soll eine oder mehrere Ziffern hinzugefügt werden, wodurch die konstuierte Nachricht m nah zu Wurzel von n also 455 liegt.
Dazu ist vorgegeben, dass die Multiplikation der Speicherwörter (4; 11) und (11; 4) fehlerhafte, pseudozufällige Werte liefern. Wir nehmen uns das Speicherwort (11; 4) vor und umwandeln es in Binärsystem, was zusammengefügt ’10110100’ ergibt. Diese binäre Zahlenfolge soll am Ende der von uns gewählten Zahl wieder vorkommen. Darum addieren wir am Anfang dieser binären Zahlenfolge eine ’1’ und enthalten ’110110100’, was dezimal ’436’ ergibt. Dann geben wir unsere m bzw. ’436’ zweimal als Eingabe und enthalten verschiedene Signaturwerte. Dadurch lässt sich sagen, dass m sich für den Angriff eignet, da es verschiedene Signatur erzeugen kann. Diese Signaturwerte werden wir auch für die weiteren Berechnungen benötigen.

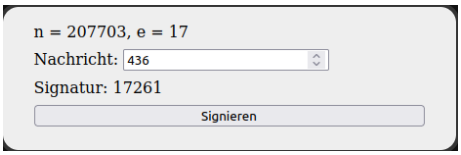

Abbildung 4: Signatur 1

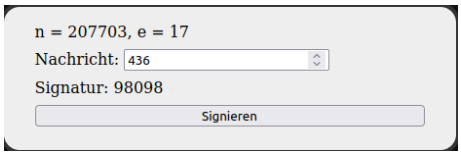

Abbildung 5: Signatur 2

Nun wollen wir n faktorisieren, um die Primzahlen p und q finden zu können. Dabei nehmen wir uns die Formeln vor: für q = ggt(s^e-m, n) und für p = n/q. Wir setzen die Zahlen die wir bereits haben ein und enthalten q = ggt(98098^{17}-436, 207703) und für p = 207703/q. Um diese Rechnungen durchzuführen implementieren wir ein Python-Code, das größte gemeinsame Teiler finden kann. Die erwähnten Rechnungen haben wir durchgeführt und am Ende zusätzlich p und q multupliziert um die Richtigkeit des Ergebnises zu kontrollieren.

```
student@Reihe4-PC2:~/Praktikum/rsa$ python3
Python 3.10.4 (main, Jun 29 2022, 12:14:53) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> n=207703
>>> e=17
>>> pow(n, 1/2)
455.74444593434157
>>> m=436
>>> def ggt(a,b):
...     while a%b != 0:
...             a, b = b, a%b
...     return b
>>> q=ggt(pow(98098, 17)-436, n)
>>> p=n//q
>>> p
907
>>> q
229
>>> p*q
207703
>>>
```

Als letztes vergleichen wir die Wahrscheinlichkeit, ein fehlerhaft berechnetes Paar (a,b) auf einem 4-BitProzessor und auf einem 64-Bit-Prozessor zu finden. Auf 4-BitProzessor kann 16 unterschiedliche Werte dargestellt werden. Es gibt also 16*16 = 256 mögliche (a,b)-Paare. Jeder Versuch könnte als 1/256 Wahrscheinlichkeit interpretiert werden. Auf 64-Bit-Prozessor gibt es 18446744073709551616 mögliche Werte und 340282366920938463463374607431768211456 (a,b)-Paare. Also ein Versuch könnte als 1/340282366920938463463374607431768211456 interpretiert werden. Also auf 64-Bit-Prozessor ist es deutlich schwerer ein (a,b)-Paar zu finden, mit dem einen Angriff durchgeführt werden kann.
Ein (a,b)-Paar zu finden, das dazu dienen kann, fehlerhafte Signaturen zu finden, ist ein großes Problem, wie der Angriff vorhin gezeigt hat. Bei dem 4-BitProzessor ist es sehr trivial. Bei 64-Bit-Prozessor beträgt die benötigte Schritte im schlimmsten Fall
log_2(340282366920938463463374607431768211456)= 129 Bits, was gegen Brute-Force-Angriffe als sicher gilt. Daher soll 64-Bit-Prozessor eingesetzt werden, aber 4 BitProzessor soll nicht eingesetzt werden.

4 Responsible disclosure

Sehr geehrte Damen und Herren,

bei einer kürzlich erfolgten Überprüfung der neuen Kryptobibliothek konnten einige Fehler gefunden werden. Die Implementierung der Bibliothek auf einer Smartcard bietet die Möglichkeit einen sogenannten Faul-Angriff durchzuführen.

Dieser Fault-Angriff kann potentiell jeden Besitzer einer Smartcard betreffen. Der Angriff besteht daraus, dass durch absichtliches herbeiführen von Berechnungsfehlern innerhalb der Smartcard, Rückschlüsse auf das, auf der Smartcard enthaltene, Geheimnis gezogen werden können.

Durch einen erfolgreich durchgeführten Angriff ist es Angreifern möglich, die Smartcard zu Simulieren und Signaturen im Namen der Smartcard auszustellen. Dies birgt das Risiko, weitere Daten an Angreifer zu verlieren oder auch nicht vertrauenswürdigen Daten zu vertrauen.

Durch die zahlreichen verschiedenen Varianten des Fault-Angriffs, ist es nahe zu unmöglich, gegen jede einen effektiven Schutz zu implementieren. Jedoch können Gegenmaßnahmen ergriffen werden, um die einfachsten und häufigsten Formen von Fault-Angriffen zu verhindern. So kann beispielsweise die Hardware so gestaltet werden, dass immer ein konstanter Stromverbrauch sichergestellt wird. Dadurch wird verhindert, durch Schwankungen im Stromverbrauch Rückschlüsse auf die Geheimnisse in der Smartcard zu ziehen. Um mögliche Lösungen zu diskutieren wird der Kontakt zu den Entwicklern der Kryptobibliothek sowie zu den Ingenieuren der Smartcard gesucht.

Da der Angriff nicht einfach im großen Rahmen durchzuführen ist und die Hardware angepasst werden sollte, wird ein erheblicher Zeitaufwand erwartet. So kann davon ausgegangen werden, dass es bis zu drei Monaten dauert, bis die notwendigen Änderungen vorgenommen wurden.

Mit freundlichen Grüßen,

Nico Steppeler und Sabri Öztürk, Berater für IT-Sicherheit

5 Zusammenfassung und Fazit

In diesem Versuch wurde gezeigt, wie durch verschiedene Verfahren RSA teilweise oder vollständig gebrochen werden kann. Durch die Analyse von drei *public key* Dateien und passender verschlüsselter Nachrichten war es möglich, den Klartext wiederherzustellen, ohne das der *private key* bekannt war. Dies ist vor allem durch die geringe Schlüssellänge möglich. Da die RSA Schlüssellänge mindestens 2048 Bit betragen sollte, sind die gegebenen Beispielschlüssel mit einer Länge von 32 Bit nicht vergleichbar mit echt Welt anwendungen. Auch wenn Computer einestages schnell genug sein könnten um auch solche Schlüssel zu berechnen.

Zudem wurden, durch einen Fault-Angriff auf eine simulierte Smartcard, die Faktoren p und q, welche nicht öffentlich sein sollten, aus der Smartcard gelesen um die enthaltene RSA Signatur erheblich zu schwächen. Mit ein wenig mehr Informationen wäre es sogar möglich gewesen, den *private key* zum Signieren in Erfahrung zu bringen.

Dieser Versuch war sehr interessant und gut gestalltet. Die Aufgaben waren teilweise nicht ganz zu verstehen, was zu verwirrungen geführt hat. So haben wir in Aufgabe 3.3.3 Schritt 2 versucht, den *private key* zu berechnen, was nicht möglich ist. Hier wäre es praktisch zu erwähnen, dass das berechnen der Faktoren p und q bereits ausreicht und als brechen zählt. Trotzdem war es interessant zu sehen, was durch verschiedene Angriffe möglich ist und auch die Wichtigkeit von mathematischen Tricks zu schätzen.

6 LLM

In diesem Versuch wurden keine Large-Language Models(LLM) verwendet. Die Texte sind alleine die Arbeit der Mitglieder der Gruppe 43 wie zubeginn angegeben.

Literatur

[1] Ruhr-Universität Bochum Lehrstuhl für Netz-und Datensicherheit. Bachelor Grundlagenpraktikum ITS: Veranstaltungsseite der Fakultät für Elektrotechnik und Informationstechnik. URL https://informatik.rub.de/studium/lehrveranstaltungen/.

[2] Ruhr-Universität Bochum Lehrstuhl für Netz-und Datensicherheit. Aufgabenstellung zum Praktikumsversuch. Bochum, 2023.

[3] MarioS. Rsa-kryptosystem. URL https://de.wikipedia.org/wiki/RSA-Kryptosystem.